
用户信息管理平台

安全漏洞挖掘与修复报告

`http://10.133.25.156:5000`

项目版本：V9.0 - SSRF服务端请求伪造专题

报告日期：2026年7月15日

今日课程：Day7:SSRF服务端请求伪造漏洞

技术栈：Python Flask / urllib / SSRF / file://协议

一、课程背景

SSRF(Server-Side Request Forgery，服务端请求伪造)是一种通过服务器发起恶意请求的漏洞。攻击者利用存在SSRF漏洞的服务器作为跳板，向内部网络发起攻击，绕过防火墙访问内网资源。

1.1 SSRF攻击原理

当Web应用需要从外部URL获取资源(如抓取网页、下载图片、调用API)时，如果未对用户输入的URL做严格校验，攻击者可以构造恶意URL让服务器去访问本不该被访问的资源。

1.2 本次新增功能

功能	路由	说明
URL抓取	/fetch-url	用户提交URL，服务端抓取并返回内容

二、漏洞分析

2.1 漏洞代码

```
@app.route("/fetch-url", methods=["POST"])
@login_required
def fetch_url():
    target_url = request.form.get("url", "")
    # 直接访问用户提交的URL
    # 不限制协议(http/https/file)
    # 不限制内网IP(127.0.0.1/10.x.x.x)
    resp = urllib.request.urlopen(target_url, timeout=10)
    content = resp.read()
    return content
```

2.2 POC验证结果

POC	请求	说明	结果
1	http://www.baidu.com	正常抓取外部网站	状态码200 ✓
2	http://127.0.0.1:5000/	SSRF: 访问本机Web服务	成功抓取到首页内容 ⚠
3	file:///etc/passwd	SSRF: 读取本地文件	读取到系统文件 ⚠

2.3 SSRF攻击场景

```
// 场景1: 扫描内网端口
POST /fetch-url url=http://127.0.0.1:3306 → 检测MySQL
POST /fetch-url url=http://127.0.0.1:6379 → 检测Redis
POST /fetch-url url=http://127.0.0.1:9200 → 检测Elasticsearch

// 场景2: 读取本地敏感文件
POST /fetch-url url=file:///etc/shadow → 读取密码文件
POST /fetch-url url=file:///etc/nginx/nginx.conf → 读取配置
POST /fetch-url url=file:///root/.ssh/id_rsa → 读取SSH密钥

// 场景3: 攻击云服务器元数据
POST /fetch-url url=http://169.254.169.254/latest/meta-data/
→ 获取云服务器临时凭证
```

三、修复方案

3.1 限制URL协议

```
# 白名单协议
ALLOWED_SCHEMES = {"http", "https"}
parsed = urllib.parse.urlparse(target_url)
if parsed.scheme not in ALLOWED_SCHEMES:
    return "不支持的协议"
```

3.2 阻止内网IP

```
import ipaddress

host = socket.gethostbyname(parsed.hostname)
private_ranges = [
    ipaddress.IPv4Network("127.0.0.0/8"),
    ipaddress.IPv4Network("10.0.0.0/8"),
    ipaddress.IPv4Network("172.16.0.0/12"),
    ipaddress.IPv4Network("192.168.0.0/16"),
]
if any(ipaddress.IPv4Address(host) in r for r in private_ranges):
    return "禁止访问内网地址"
```

3.3 URL白名单

```
# 只允许访问特定域名
ALLOWED_DOMAINS = ["example.com", "api.example.com"]
if parsed.hostname not in ALLOWED_DOMAINS:
    return "不允许的域名"
```

3.4 修复前后对比

对比项	修复前	修复后
协议限制	无限制(file://可用)	仅允许http/https
内网限制	无限制(可访问127.0.0.1)	禁止访问内网IP
DNS解析	不验证	解析后检查IP
超时	10秒	保留超时设置

四、总结

SSRF漏洞的核心原因是服务器盲目信任用户输入的URL。攻击者可利用file://协议读取服务器文件，或利用内网IP绕过防火墙访问内部服务。

安全开发原则：

1. 严格限制URL协议，只允许http/https
2. DNS解析后检查目标IP是否为内网地址
3. 使用URL白名单或域名白名单
4. 设置合理的超时时间防止慢速攻击
5. 对返回内容做长度限制

— 报告完 —

报告日期: 2026-07-15 | 课程: SSRF服务端请求伪造漏洞修复